

DSC 102- Problems in Map Reduce

1. Suppose you are given a large dataset with 50 numeric and 9 categorical features (domain size of 50 each). The HDFS file size is 3 TB.
 - a. Write pseudocode for (or just describe precisely in prose) MapReduce job(s) to compute this dataset's correlation matrix. Hint: Do it as 2 separate MapReduce jobs, although 1 suffices.
 - b. What is the rough total disk I/O cost of the above in TB? Include both reads and writes of intermediate data and output.

Ans: Suppose we one-hot encode all categorical features into 50-dimensional 0-1 vectors. Then the total number of numerics is $50 + 9 \times 50 = 500$. So, the correlation matrix is of size 500×500 , which is 250,000 cells. Even with float64, it is only 2MB. So, we will use this as our aggregation state for Mappers to send to Reducer.

- a. **Input Split:** *Shard row-wise as usual. - Our approach to compute correlation matrix uses 2 MR jobs: the first to compute the per-feature means and stdevs; and the second to use those to finish the correlation computations.*

First Map() *reads tuple, converts each categorical feature to its respective one-hot encoded vector to stitch together full 500- dimensional numeric vector, and computes the "sufficient statistics" needed for means and stdevs, viz., running example count and running (x, x^2) for each feature; emit all those stats as one long vector as value with a single global dummy key.*

First Reduce() *aggregates all stats vectors to emit total example count and 2 vectors: means of all features, stdevs of all features.*

Second Map() *reads tuple, gets the 500-dimensional numeric vector again as before and then emits as stats a 500×500 matrix representing pairwise products for the aggregation needed for the numerator of the Corr matrix formula below.*

Second Reduce() *just adds up these individual matrices and divides all cells by total example count and the respective pairs of stdevs obtained from the first MR job.*

$$\text{Corr}(A, B) = E[(A - \mu_A)(B - \mu_B)] / (\sigma_A \sigma_B)$$

- b. 2 filescans amount to 6 TB. Intermediate data after the first job has 500×2 numbers; output is 500×500 matrix. Assuming all numbers are float64, these amount to mere ~ 2 MB extra. So, overall I/O cost is still roughly 6 TB.
-

2. You are given a large $n \times d$ matrix on HDFS that is uniformly sharded column-wise across k workers.
- Write a MapReduce job (pseudocode or precise prose) to obtain the **row sums vector** of the matrix (aka rowSums in R). It should be scalable along the number of columns.
 - What is the minimum possible communication cost (in big O notation of n , d , and k) to get the row sums vector using MapReduce?

Ans:

- a. *MR Solution*

InputSplit: Same as the column-wise shards given.

Map(): Do a vector sum of all the columns in the input shard. Emit the partial rowSums vector as value with a single global dummy key.

Reduce(): Gets an iterator with all partial rowSums vectors from mappers; vector sum of all of them; emit that as final output with no key.

- b. $O(nk)$ because each of the k workers send a state of size $O(n)$; note that this cost is independent of d
-

3. Given a large $n \times d$ matrix on HDFS propose a sharding and write a MapReduce job to obtain its **columnar sums vector** (aka colSums in R). The layout and job should both be scalable along both the number of rows and number of columns.

Ans:

InputSplit: Tile-wise sharding with tile shape as a fixed known hyperparameter (e.g., 1000×1000) and one tile per shard. So, each shard has (tile-row, tile-col) indices associated with it in the whole matrix.

Map(): Do a colSums over all the rows in the input tile/shard. Emit this partial vector as value with the tile's tile-col index value as the key.

***Reduce():** Key is a tile-col index in the matrix. Gets an iterator with all partial colSums vectors from mappers that got a tile that had that tile-col index. Vector sum of all of the partial vectors; emit that as final output value with ColSums for all the columns that tile-col as the key. So, the output is itself written out in a sharded form by multiple reducers.*

4. You are given a large table **Ratings (RID, Stars, Date, MovieID, UserID)** from a recommender system stored on HDFS. You want to know the **average rating (Stars) for each movie**. Propose a sharding and write a MapReduce job (pseudocode or precise prose) to obtain the above. It should be scalable along the both the number of tuples and the number of movies.

Ans:

***InputSplit:** Tuple-wise sharding, with multiple tuples per shard but shards are not so big as to not fit in a mapper's memory.*

***Map():** Create a hash table data structure with key being MovieID and value being the 2-tuple (partial sum of Stars, partial count). Go through rows in the input shard to populate this hash table. At the end, for each record in the hash table do an emit with key being its MovieID and value being its 2-tuple.*

***Reduce():** Key is a MovieID value. Gets an iterator with all partial 2-tuples of sums and counts. Vector sum of all of them to get global Sum and global count of that MovieID; divide sum by count to get average; emit that as final output value with that MovieID as the key. So, the output is itself written out in a sharded form by multiple reducers.*

5. Consider the same table Ratings from the previous question. Suppose you are now given it is a tuple-wise sharded file on HDFS. You aim to **binarize the Stars attribute** as part of your ML data preparation to cast the recommendation/ratings prediction task as binary classification. You want to see the histogram of Stars values. Write a MapReduce job (pseudocode or precise prose) to get this. It should be scalable along the number of tuples.

Ans: No histogram bins or widths given; so, just get count of each unique value

InputSplit: Same as the tuple-wise shards given, with multiple tuples per shard but shards are not so big as to not fit in a mapper's memory.

Map(): Create a hash table data structure with key being Stars and value being the partial count. Go through rows in the input shard to populate this hash table. At the end, emit this hash table as value with a single global dummy key.

Reduce(): Gets an iterator with all partial hash tables from mappers; unify the hash tables into one based on the keys to add up the partial counts by key/each Stars value—this gives you the full histogram. Emit that as the final output with no key.

6. Continuing from the previous question, now based on the histogram you decide to binarize the Stars attribute with a threshold of 3.5, i.e., Stars ≥ 3.5 is set to +1; Stars < 3.5 is set to -1. Write a MapReduce job (pseudocode or precise prose) to do this. It should be scalable along the number of tuples.

Ans: This is a Map-only job because no global agg. is needed. **InputSplit:** Same as previous question.

Map(): For each row in the input shard, copy its tuple and edit Stars value as per thresholding rule given, and emit it as value with the same tuple ID as key as in the input.
